11/05 발표 · 정민지 파트 슬라이드별 상세 해설

슬라이드 3~13장 · 장마다 실험 이유, 세팅, 방식, 측정, 결과, 발표 한마디, 예상 질문

이 문서 사용법

> 11/05 발표의 정민지 파트(3~13장)를 장마다 **① 실험 이유 → ② 실험 세팅 → ③ 실험 방식 → ④ 측정 방식 → ⑤ 실험 결과 → 발표 한마디 → 예상 질문** 순서로 정리했습니다.

전체 이야기 흐름

3 · 4 RQ-VAE로 SID 만들기 → 5 · 6 Transformer로 추천 → 7 · 8 정성평가 → 9 ~ 12 정량평가 → 13 NRMS로 추천 정보 확인

RQ-VAE로 기사마다 SID를 만들고(3~4장), SID로 추천 모델을 학습해 성능을 확인한 뒤(5~6장), 그 SID가 기사 내용을 잘 담는지 사람 · 단어 · 임베딩 · 이웃 네 방향으로 확인하고(7~12장), 마지막으로 표준 추천 모델에서 추천 정보가 남는지 봤습니다(13장).

꼭 기억할 숫자

무엇	숫자	장
uniqueness loss로 줄어든 충돌	31.56% → 24.39%	4
Transformer 탐색	6단계 · 84 run, val Top-1 27.06%	5
최종 추천 V1 / + level 가중치	27.07% / 29.46%	6
사람 판정 “높음” 비율	0% → 27% → 77%	7
본문 TF-IDF 평균	0.029 → 0.080 → 0.248	9
원본 임베딩 cosine	0.799 → 0.818 → 0.847 → 0.907	10
자기 복원 평균	0.9275	11
KNN 같은 c1 / c1c2 (무작위 대비)	70.85% / 28.20% (4.5× / 94×)	12

NRMS E5 / recon / SID	29.76% / 30.28% / 26.09%	13

공통 범위 · SID 품질 평가(7~12장)는 RQ-VAE 학습에 쓰인 train 기사 기준입니다(11장은 train + validation). 추천 성능(5 · 6 · 13장)은 모두 validation 앞쪽 절반으로 고르고 뒤쪽 절반(test 122,668문제)으로 한 번 잰 숫자입니다.

발표 전 고칠 것 1개 · 6장 결과 분석의 "TIGER 17.36% < 무작위 …" 한 줄은 TIGER를 이번 발표에서 빼기로 했으니 지웁니다(`reports/슬라이드_최종추천성능_TIGER제외.png` 로 교체 가능).

3장 · RQ-VAE · 어떤 파라미터를 어떻게 탐색했나

RQ-VAE 설계



11/05 발표 슬라이드 3

> 기사 임베딩을 SID로 바꾸는 RQ-VAE를, **복원 정확도뿐 아니라 SID 품질 지표 5개**를 함께 보고 골랐습니다. 여기서 뽑힌 기본 모델이 WD-001입니다.

1 실험 이유

- SID는 이후 모든 실험(추천 모델 입력, 품질 평가)의 기본 단위라, SID를 만드는 RQ-VAE 설정이 결과 전체를 좌우합니다.

- 복원 loss만 낮다고 좋은 SID가 아닙니다. 서로 다른 기사가 같은 코드를 받으면(충돌) 추천 모델이 둘을 구분하지 못하고, 코드가 일부에만 몰리면 표현력이 떨어집니다.
- 그래서 “잘 복원하나 + 기사를 잘 구분하나 + 같은 사건을 묶나 + 코드를 고루 쓰나 + prefix에 의미가 있나”를 함께 봐야 했습니다.

2 실험 세팅

항목	값
입력	기사 하나의 E5 임베딩 768차원 (제목 + 부제로 생성)
학습 데이터	train 기사 9,738개
검증 데이터	validation 기사 3,122개 (복원 loss 계산용)
SID 구조	c1 = 편집 카테고리 25개 (학습하지 않고 고정) · c2 = 128개 · c3 = 512개 · c4 = 같은 c1c2c3 안의 순번(충돌 구분용)
탐색한 변수	코드북 크기(Q2 64~512, Q3 128~1024) · 잠재 차원과 hidden 구조 · 양자화 방식(STE · Gumbel · Rotation, 정규화 여부) · loss 가중치 · 학습률 · batch · weight decay
최종 기본 설정 (WD-001)	latent 128 · hidden 512 · C1 25 / C2 128 / C3 512 · STE · 정규화 없음 · loss = rec 1 + codebook 0.25 + commitment 0.1

3 실험 방식

1. 변수를 한 번에 다 바꾸지 않고 단계별로 하나씩(또는 같이 움직이는 것끼리) 바꿔 가며 학습했습니다. 마지막 단계가 weight decay(WD)이고, 그 001번 설정이 WD-001입니다.
2. 각 학습 run 안에서는 **10 epoch마다 validation 기사 3,122개의 복원 loss**를 재고, 가장 낮은 시점의 checkpoint(best_rec)를 저장했습니다. 최소 200 epoch은 돌리고, 20번 연속(=200 epoch) 0.001 이상 개선이 없으면 멈춥니다(최대 epoch 설정값 1,283).
3. 설정끼리 비교할 때는 아래 지표 5개를 함께 보고 판단했습니다(결과표를 LLM에 보여 주고 함께 판단).

4 측정 방식

지표	방향	어떻게 재나
Valid rec loss	↓	validation 기사의 E5를 SID로 복원했을 때 원본과의 오차 (MSE)

C123 collision	↓	서로 다른 기사가 같은 (c1, c2, c3)를 받은 비율 (4장에 계산 예시)
C2 event consistency	↑	전처리에서 같은 사건(event_id)으로 묶인 기사들이 같은 c2를 받은 비율
Q2 / Q3 usage	↑	코드북 128 / 512개 중 실제로 한 번 이상 쓰인 코드 수
ΔC2 / ΔC3	↑	prefix를 한 자리 더 공유할 때 의미 유사도가 얼마나 더 오르나

Valid rec loss 계산 (`modules/loss.py`, `train_rqvae.py`)

```
① 기사 하나: 768칸의 차이 제곱을 모두 더함 → 숫자 1개
   rec(기사) = Σₖ (x̂[k] − x[k])²     x = 원본 E5 (길이 1), x̂ = 복원 벡터
   예) h3 빙에고르 폭설: (0.015−0.008)² + (0.026−0.032)² + (−0.032+0.046)² + … = 0.1418
② validation 기사 3,122개의 평균 → validation rec loss  (UNI epoch 180 = 0.152458)
③ 10 epoch마다 계산, 지금까지 최저보다 0.001 넘게 낮으면 checkpoint_best_rec.pt 저장
```

칸 방향으로는 합, 기사 방향으로는 평균입니다. 이 값으로 epoch을 고른 이유: RQ-VAE의 목적(E5 정보를 잃지 않고 압축)을 직접 재는 값이고, 정답 라벨 없이 validation만으로 자동 비교할 수 있기 때문입니다. 한계: 복원이 가장 좋은 epoch이 SID 품질(충돌 · 일관성)도 가장 좋다는 보장은 없습니다.

---

5 실험 결과

단계별 탐색 끝에 **WD-001**을 기본 RQ-VAE로 정했습니다. 다만 WD-001은 충돌이 31.56%로 높아, 다음 장에서 이걸 줄이는 uniqueness loss를 추가합니다.

> **발표 한마디** · SID를 만드는 RQ-VAE는 복원만 잘하면 되는 게 아니라 기사 구분 · 사건 묶음 · 코드 사용 · prefix 의미까지 봐야 해서, 이 다섯 지표로 설정을 골랐습니다.

예상 질문과 답

Q. WD-001은 무슨 뜻인가요?

A. 단계별 탐색의 마지막 단계(weight decay)에서 001번 설정입니다. 이후 모든 실험의 기본 RQ-VAE이고, 여기에 uniqueness loss를 더한 것이 UNI입니다.

Q. c1은 왜 학습하지 않나요?

A. 사람이 해석할 수 있도록 편집 카테고리 25개로 고정한 설계입니다. 그래서 E5로는 비슷한데 카테고리가 다른 기사는 c1부터 갈립니다(12장 KNN에서 약 29%).

Q. 설정은 어떻게 최종 결정했나요?

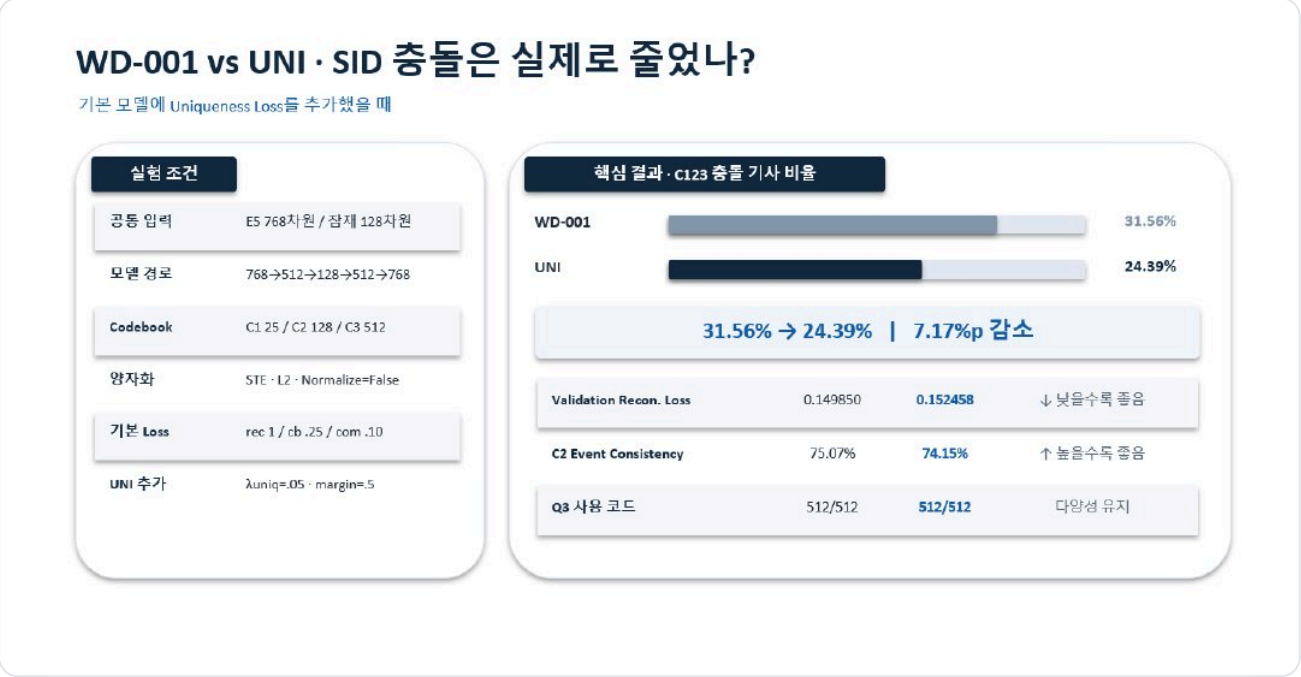
A. 두 단계입니다. ① 어떤 설정을 쓸지는 지표 5개를 함께 보고 판단했습니다. ② 각 설정의 학습 run에서 몇 번째 epoch을 쓸지는 validation 복원 loss가 가장 낮은 시점으로 정했습니다 (UNI는 epoch 180).

Q. 코드가 일부에만 몰리는 문제는요?

A. K-means로 코드북 초기값을 잡고 시작했고(seed 42), 최종 모델은 Q3 512개를 모두 씁니다(4장).

4장 · WD-001 vs UNI · SID 충돌은 실제로 줄었나

RQ-VAE 결과



11/05 발표 슬라이드 4

> uniqueness loss를 더하자 **충돌 기사 비율이 31.56% → 24.39%(−7.17%p)**로 줄었고, 대신 복원과 사건 일관성은 아주 조금 나빠졌습니다.

1 실험 이유

- WD-001은 기사 3개 중 1개꼴(31.56%)로 다른 기사와 같은 (c1, c2, c3)를 받았습니다.
- 같은 SID를 받은 기사는 Transformer 점수(c1~c3로 계산)가 똑같아서 추천 모델이 구분할 수 없습니다. 그래서 충돌을 줄이는 것이 목표였습니다.

2 실험 세팅

항목	WD-001	UNI
공통 입력 · 구조	E5 768 → 512 → 128 → 512 → 768, C1 25 / C2 128 / C3 512, STE	같음
loss	rec 1 + codebook 0.25 + commitment 0.1	+ **uniqueness loss** (λ 0.05, margin 0.5)
checkpoint	best_rec	best_rec (**epoch 180**)

3 실험 방식

1. WD-001과 모든 설정을 같게 두고 uniqueness loss 하나만 더해 다시 학습했습니다.
2. uniqueness loss: 같은 (c1, c2, c3)에 들어간 기사 쌍의 잠재 벡터 cosine이 0.5를 넘으면, 넘은 만큼 벌점을 줍니다. 너무 비슷하게 뭉친 기사들을 밀어내 서로 다른 코드를 받게 합니다.
3. 두 모델로 기사 전체의 SID를 만든 뒤 같은 지표로 비교했습니다.

4 측정 방식

지표	대상 데이터	계산
c123 충돌 기사 비율	train + validation 기사 12,860개	같은 (c1,c2,c3)를 가진 기사가 2개 이상인 그룹에 속한 기사 수 ÷ 전체 기사 수
Validation 복원 loss	validation 기사 3,122개	E5와 복원 벡터의 오차
C2 event consistency	12,860개	같은 사건 기사들 안에서 가장 많이 나온 c2의 비율 평균
Q3 사용 코드	12,860개	c3 512개 중 쓰인 코드 수

5 실험 결과

지표	WD-001	UNI	더 좋은 쪽
c123 충돌 기사 비율 ↓	31.56%	24.39%	UNI (−7.17%p)

Validation 복원 loss ↓	0.149850	0.152458	WD-001 (+0.0026)
C2 event consistency ↑	75.07%	74.15%	WD-001 (−0.92%p)
Q3 사용 코드	512 / 512	512 / 512	같음

충돌은 줄고 복원 · 일관성은 소폭 악화되는 상충관계입니다. 충돌 감소 폭이 훨씬 커서 이후 모든 실험은 UNI SID를 씁니다.

발표 한마디 · uniqueness loss 하나만 추가해서 충돌 기사를 31.56%에서 24.39%로 줄였고, 복원과 사건 일관성은 거의 유지됐습니다.

충돌 비율은 이렇게 계산합니다

충돌 기사 비율 = c1c2c3가 같은 다른 기사가 있는 기사 수 ÷ 전체 기사 수. 같은 (c1, c2, c3) 끼리 묶고, 2개 이상 묶음의 기사를 모두 셉니다. c4는 보지 않습니다(c4는 같은 c1c2c3 안의 순번 = 충돌 표시).

작은 예시 (기사 10개)	묶음	크기
A, B, C	20-122-378	3 ← 충돌
F, G	12-22-288	2 ← 충돌
D / E / H / I / J	각자 다름 (D 21-117-485, E 21-117-147은 c3가 달라 충돌 아님)	1씩
충돌 기사 = A, B, C, F, G = 5개 → 5 ÷ 10		50%

실제 UNI (12,860개)	묶음 수	기사 수
혼자인 묶음	9,723	9,723
2개 / 3개 / 4~17개 묶음	893 / 188 / 148	1,786 / 564 / 787
2개 이상 합계 → 3,137 ÷ 12,860	1,229	24.39%

실제 예: 20-122-378 묶음 = Cecilie 20세(train, c4 0) · Cecilie 21세(train, c4 1) · validation 기사 1개(c4 2) → 세 기사 모두 충돌로 셈(train · validation이 섞여도 셈). 가장 큰 묶음이 17개라 max c4 = 16. 코드에 있는 다른 지표 `c123_collision_rate` = 1 − 묶음 수 ÷ 기사 수 = 14.84%와 섞지 않도록 주의합니다(슬라이드는 기사 비율).

예상 질문과 답

Q. 24%도 여전히 많지 않나요?

A. c4(같은 c1c2c3 안 순번)로 기사 구분은 됩니다. 다만 추천 점수는 c1~c3만 쓰므로, 1P4N 후보를 만들 때 정답과 c1c2c3가 같은 오답은 뺐습니다. 더 줄이려면 λ를 키우거나 c3를 늘려야 하는데 복원 · 일관성과의 상충이 커집니다.

Q. 충돌 비율을 다른 방식으로 세면요?

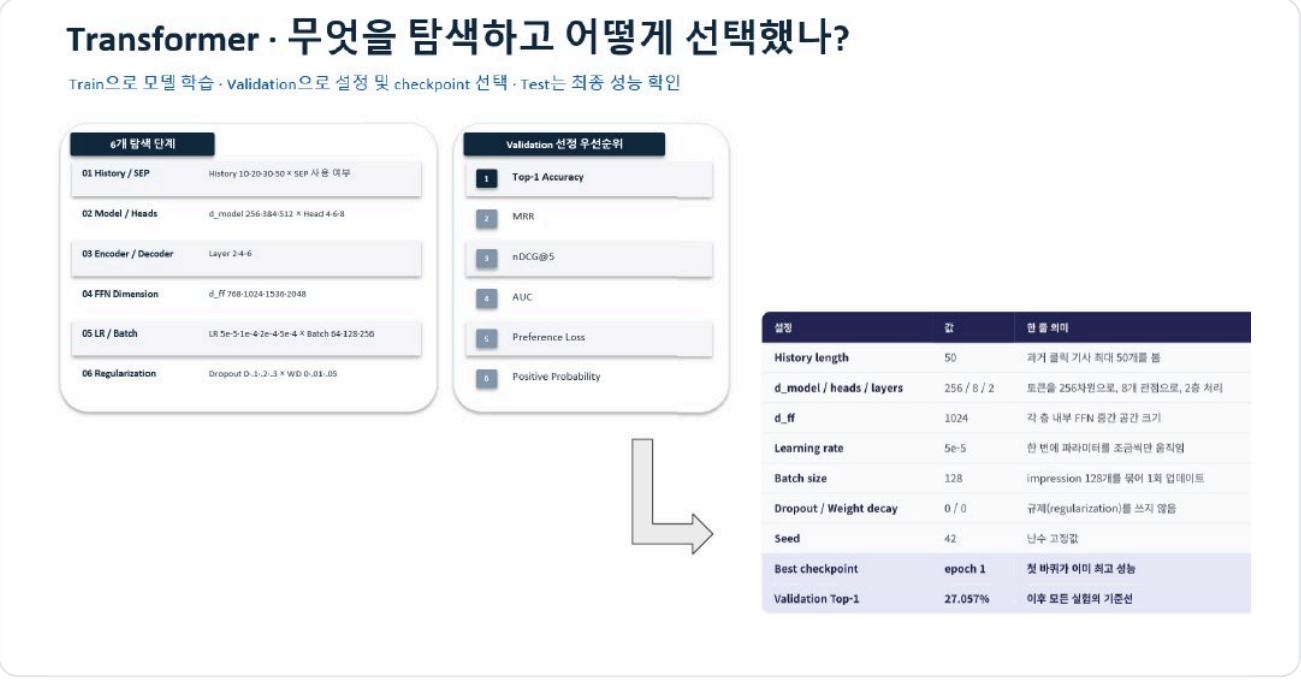
A. "1 − 그룹 수 ÷ 기사 수"로 세면 14.84%입니다. 슬라이드의 24.39%는 "충돌 그룹에 속한 기사" 기준입니다.

Q. 사건 일관성 74%는 무슨 뜻인가요?

A. UNI는 학습 때 c2를 기사마다 따로 고르기 때문에 같은 사건이라도 c2가 갈릴 수 있습니다. 원래 설계(사건 단위 c2)는 일관성 100%가 목표라, 여민 님이 이 부분을 개선 중입니다.

5장 · Transformer · 무엇을 탐색하고 어떻게 선택했나

Transformer 설계



11/05 발표 슬라이드 5

> 조합 96,768가지를 **6단계 · 84 run**으로 줄여 validation으로만 골랐고, 최종 V1은 history 50 · 256 / 8 head / 2층 · lr 5e-5 · 규제 없음입니다.

1 실험 이유

- SID를 입력으로 받는 추천 모델(V1)의 구조와 학습 설정을 정해야 했습니다.
- 조합을 전부 돌릴 수 없고, test를 보고 고르면 결과가 부풀려지므로 validation만으로 체계적으로 고르는 규칙이 필요했습니다.

2 실험 세팅

항목	값
모델	T5 형태 encoder-decoder Transformer
입력	사용자가 읽은 최근 기사(최대 50개)의 SID 토큰 c1 c2 c3 c4를 이어 붙인 열 + 후보 기사 5개의 SID
출력 · 점수	decoder가 후보의 c1 → c2 → c3를 한 자리씩 확률로 평가: S = log P(c1) + log P(c2|c1) + log P(c3|c1,c2)
학습 목표	후보 5개의 S에 softmax → 정답 후보의 −log 확률 (listwise CE, NRMS와 같은 방식)
데이터	1P4N: train 232,874문제 / validation half 122,549문제 (test 122,668은 탐색에 안 씀)
학습 규칙	최대 30 epoch, validation Top-1이 5번 연속 안 오르면 정지(patience 5), best epoch 저장

3 실험 방식

단계	함께 탐색	후보	run 수	선정
1	History × SEP	10 · 20 · 30 · 50 × 사용 여부	8	50 / 미사용
2	d_model × heads	256 · 384 · 512 × 4 · 6 · 8 (나누어떨어지는 7쌍)	14	256 / 8
3	layers	2 · 4 · 6	6	2
4	d_ff	768 · 1024 · 1536 · 2048	8	1024
5	LR × batch	5e-5 · 1e-4 · 2e-4 · 5e-4 × 64 · 128 · 256	24	5e-5 / 128
6	dropout × weight decay	0 · 0.1 · 0.2 · 0.3 × 0 · 0.01 · 0.05	24	0 / 0
합계			84	

- 같이 움직이는 파라미터(예: lr과 batch)는 같은 단계에서 조합으로 탐색했습니다.
- 단계마다 상위 2개를 다음 단계로 넘겨, 한 번에 하나만 고를 때보다 잘못된 길로 빠질 위험을 줄였습니다.
- 마지막 상위 3개 설정은 seed 42 · 123 · 2026으로 다시 확인했습니다.

---

4 측정 방식

설정끼리는 아래 우선순위로 비교했습니다. 앞 지표가 허용 범위 안에서 같으면 다음 지표로 넘어갑니다(Top-1 허용 범위 0.5%p). 모두 같으면 파라미터 수 → epoch 시간 → GPU 메모리가 작은 쪽입니다.

순위	지표	뜻
1	Top-1 Accuracy	후보 5개 중 1등이 정답인 비율
2	MRR	정답의 순위 역수 평균 (1등 1, 2등 0.5 ...)
3	nDCG@5	정답이 위에 있을수록 높은 순위 점수
4	AUC	정답 점수가 오답 점수보다 높은 쌍의 비율
5	Preference Loss	정답 · 오답 점수 차이로 계산한 loss (낮을수록 좋음)
6	Positive Probability	정답 후보에 준 평균 확률

한 설정 안에서 몇 번째 epoch을 쓸지는 validation Top-1 하나로 골랐습니다.

Validation Top-1 계산

```
데이터: EB-NeRD validation 주 클릭 기록을 시간순으로 반으로 나눔
        앞쪽 절반 = validation half 122,549문제 (선택용) · 뒤쪽 절반 = test
122,668문제 (보고용)
문제 1개 = 1P4N 한 줄 = (impression 1개, 그 impression에서 클릭한 기사 1개) + 클릭 안
한 기사 4개
        한 impression에서 2개 클릭 → 문제 2개 · 입력 history = 그 시점까지 읽은 기사
(최근 50개)
문제마다: 후보 5개 점수 S 계산 → 가장 큰 후보 = 예측 → 실제 클릭이면 1, 아니면 0
validation Top-1 = 맞힌 문제 수 / 122,549
   예) 27.057% ≈ 33,158문제 정답 (무작위 20% ≈ 24,510문제)
```

epoch이 끝날 때마다 계산해 가장 높은 epoch을 저장합니다. test Top-1도 같은 계산을 test 122,668문제에 한 번만 한 값입니다(V1 27.067% = 33,202문제 정답).

5 실험 결과

설정	값
History · SEP	최근 50개 · SEP 없음
d_model / heads / layers	256 / 8 / 2
d_ff	1024
Learning rate · batch	5e-5 · 128 (AdamW)
Dropout · weight decay	0 · 0
Seed	42
Best checkpoint	epoch 1
Validation Top-1	27.06%

best epoch이 1인 이유: 첫 epoch 뒤로는 train loss만 내려가고 validation은 떨어집니다. test 정답의 98% 이상이 train에서 클릭된 적 없는 새 기사라, 오래 학습할수록 train 기사에 맞춰지는 과적합이 빠르게 옵니다.

발표 한마디 · 조합이 너무 많아 6단계로 나눠 84번 돌렸고, 모든 선택은 validation으로만 했습니다. 최종 설정은 validation Top-1 27.06%입니다.

예상 질문과 답

Q. 왜 grid search를 다 하지 않았나요?

A. 96,768 조합은 불가능해서, 같이 움직이는 파라미터는 같은 단계에서 조합으로 보고 나머지는 단계별로 나눴습니다. 단계마다 2개를 남겨 위험을 줄였습니다.

Q. test로 고른 것은 없나요?

A. 없습니다. 모든 선택은 validation 앞쪽 절반으로 했고 test는 최종 설정을 정한 뒤 한 번만 쟀습니다.

Q. 슬라이드의 "Validation Top-1 27.057% · 이후 모든 실험의 기준선"은 test로 해야 하는 것 아닌가요?

A. 설정을 고르는 단계라 validation이 맞습니다. test로 고르면 test 성적이 부풀려집니다. 다만 "기준선"은 **이후 설정 비교(validation)의 기준선**이라는 뜻이고, 다른 모델과 비교하는 최종

성적은 test 27.07%(6장)입니다. 슬라이드 문구를 “설정 선택 기준 (test 성적은 6장 27.07%)”으로 바꾸면 오해가 없습니다.

Q. validation 27.06%와 test 27.07%가 비슷한데 같은 데이터인가요?

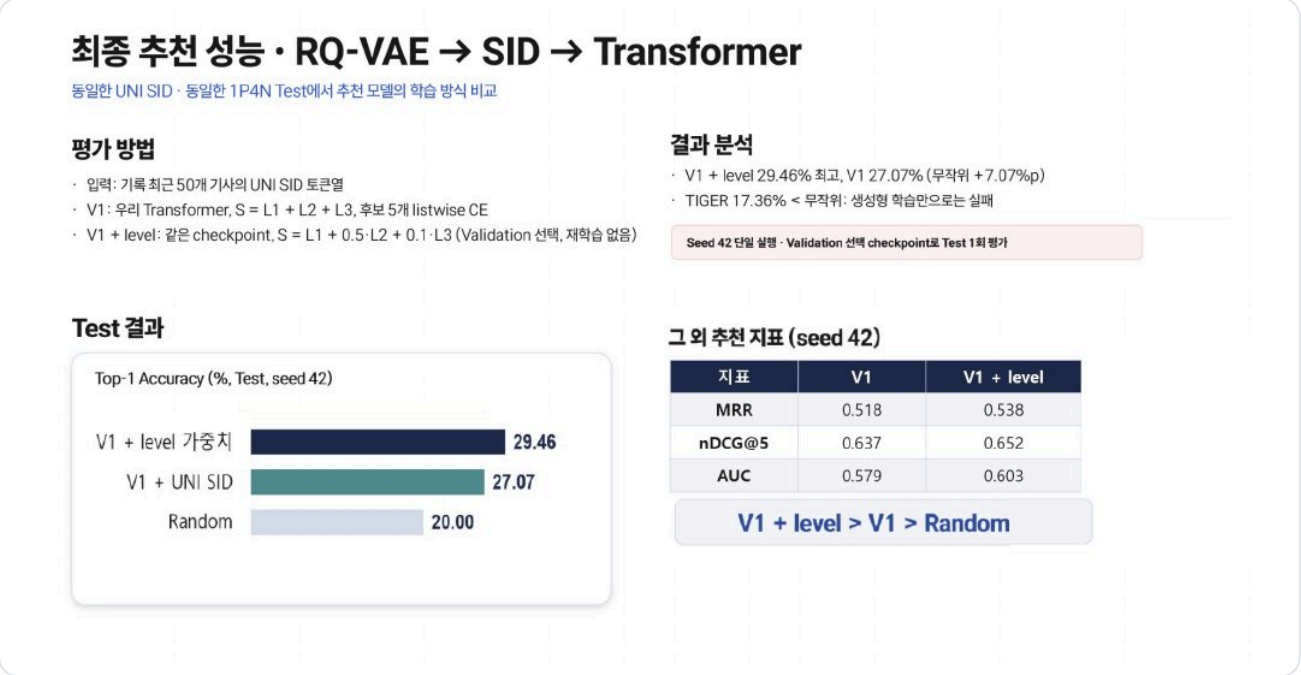
A. 아닙니다. validation은 validation 주의 앞쪽 절반, test는 뒤쪽 절반입니다. 우연히 비슷하게 나왔습니다.

Q. best epoch 1이면 학습이 안 된 것 아닌가요?

A. 첫 epoch에 이미 validation 최고가 나오고 이후로는 과적합입니다. 새 기사 위주의 뉴스 데이터 특성입니다.

6장 · 최종 추천 성능 · RQ-VAE → SID → Transformer

최종 결과 Test



11/05 발표 슬라이드 6

> UNI SID를 V1에 넣은 최종 추천 성능은 **test Top-1 27.07%**, 점수식에서 c3 비중을 줄이면 **29.46%**로 무작위(20%)보다 확실히 높습니다.

---

1 실험 이유

- RQ-VAE가 만든 SID를 추천 모델까지 통과시켰을 때 실제로 추천이 되는지, 처음부터 끝까지 확인하는 실험입니다.

- V1의 점수식 S는 c1 · c2 · c3를 똑같이 더하는데, 자리마다 쓸모가 다를 수 있어 가중치를 바꿔 보았습니다.

2 실험 세팅

항목	값
SID	UNI (RQ-VAE epoch 180)
모델	5장에서 고른 V1 설정 그대로, seed 42
데이터	train 232,874 학습 → validation half 122,549로 best epoch(1)과 level 가중치 선택 → **test 122,668문제로 한 번 채점**
test 만드는 법	EB-NeRD는 정답이 있는 test가 없어서, validation 주 클릭 기록을 시간순으로 반으로 나눠 뒤쪽 절반을 test로 사용
후보	1P4N: 클릭 1개 + 같은 노출에서 클릭 안 한 4개, 후보 순서 섞음, 정답과 c1c2c3 같은 오답 제외

3 실험 방식

1. **V1**: test 문제마다 후보 5개의 S = L1 + L2 + L3를 계산해 가장 큰 후보를 추천합니다. (L1 = log P(c1), L2 = log P(c2|c1), L3 = log P(c3|c1,c2))
2. **V1 + level 가중치**: 같은 checkpoint로 S′ = L1 + α2·L2 + α3·L3를 씁니다. 재학습은 없습니다.
3. α는 validation half에서 α2 7개 {0, 0.25, 0.5, 0.75, 1.0, 1.25, 1.5} × α3 6개 {0, 0.1, 0.25, 0.5, 0.75, 1.0} = 42가지를 비교해 **(α2, α3) = (0.5, 0.1)**을 고른 뒤 test에 한 번 적용했습니다.

4 측정 방식

지표	계산
Top-1	1등 후보가 정답인 문제 비율 (무작위 1/5 = 20%)
MRR	정답 순위의 역수 평균
nDCG@5	정답이 높은 순위일수록 큰 점수
AUC	정답 점수 > 오답 점수인 (정답, 오답) 쌍 비율 (무작위 0.5)

5 실험 결과

모델	Top-1	MRR	nDCG@5	AUC
V1 + level 가중치	29.46%	0.538	0.652	0.603
V1 + UNI SID	27.07%	0.518	0.637	0.579
Random	20.00%	—	—	0.500

- V1은 무작위보다 +7.07%p, level 가중치로 +2.39%p 더 오릅니다. 네 지표 모두 같은 방향입니다.
- c3 비중을 0.1로 줄일 때 가장 좋았습니다. c3는 기사 수준의 세부 코드라 처음 보는 새 기사에는 잘 일반화되지 않고, c1 · c2(카테고리 · 주제)는 잘 일반화됩니다.

발표 한마디 · RQ-VAE부터 Transformer까지 통과시킨 최종 성능은 27.07%, c3 비중을 줄이면 29.46%입니다. c3는 새 기사에 일반화가 어려워서 비중을 줄일수록 좋아졌습니다.

예상 질문과 답

Q. level 가중치를 test 보고 고른 건 아닌가요?

A. 아닙니다. validation half에서 42가지 중 골랐고, test에서도 +2.39%p가 유지됐습니다.

Q. seed가 하나인데 믿을 수 있나요?

A. V1은 seed 42 한 번입니다. 1%p 안쪽 차이는 같은 수준으로 봐야 하지만, 무작위 대비 +7%p와 level 가중치 +2.4%p는 그보다 큽니다.

Q. 13장 NRMS recon(30.28%)이 더 높은데요?

A. recon은 RQ-VAE의 768차원 벡터까지 쓰는 경우라 입력이 다릅니다. SID 번호만 쓰는 조건으로 맞추면 NRMS 26.09%, V1 27.07%, V1 + level 29.46%로 V1이 높습니다. V1에도 RQ-VAE 코드북을 넣어 보는 건 다음 실험 후보입니다.

Q. TIGER와는 비교 안 했나요?

A. 공개 재현 구현을 돌려 봤지만(17.36%) 학습 · 평가 방식이 달라 공정한 비교 방법을 먼저 정해야 해서 이번 발표에서는 뺐습니다. 어떻게 비교하면 좋을지 여쭤보고 싶습니다.

7장 · 정성평가 · 평가 설계와 판정 기준

SID 품질 정성



11/05 발표 슬라이드 7

> 사람이 기사를 직접 읽어 보니 SID를 더 공유할수록 내용이 비슷했습니다. “높음” 비율 **0% → 27% → 77%**.

---

1 실험 이유

- 숫자 지표가 놓칠 수 있는 “실제로 읽어 봤을 때 비슷한가”를 확인하려는 실험입니다.
- 같은 사건인지가 아니라 **기사 전체 내용이 얼마나 비슷한지**를 봤습니다.

---

2 실험 세팅

조건	정의	표본
Prefix-3	c1 · c2 · c3가 모두 같은 기사들	30그룹: 크기 2~3개 15그룹, 4~5개 10그룹, 6~8개 5그룹. 그룹 안 기사는 전부 읽음
Prefix-2	c1 · c2는 같고 c3는 다른 기사들	기사 4~10개짜리 15그룹
Different	SID가 전부 다른 기사 쌍 (대조군)	30쌍

대상은 train 기사입니다. 큰 그룹에서 의미가 무너지는지 보려고 큰 그룹을 일부러 많이 뽑았습니다.

3 실험 방식

1. 조건마다 그룹(또는 쌍)을 뽑습니다.
2. 덴마크어 원문(제목 · 부제 · 본문)을 읽고 그룹 전체가 얼마나 비슷한지 3단계로 판정합니다.
3. AI가 1차 판정을 하고 사람이 교차 검토해서 확정합니다.

4 측정 방식

판정	기준
높음	핵심 내용 · 주제 정보가 매우 비슷함
중간	상위 주제는 같지만 세부 내용이 다름
낮음	일부 단어만 겹치거나 실질적인 연관이 약함

5 실험 결과

조건	높음	중간	낮음	"높음" 비율
Different (30쌍)	0	0	30	0%
Prefix-2 (15그룹)	4	5	6	27%
Prefix-3 (30그룹)	23	5	2	77%

SID를 공유할수록 "높음"이 늘고 "낮음"이 줄어듭니다. SID가 아무 관계 없는 기사(Different)는 모두 낮음입니다.

> **발표 한마디** · SID가 같은 기사를 직접 읽어 보니 Prefix-3는 77%가 "높음"이었고, SID가 다른 기사는 모두 "낮음"이었습니다.

예상 질문과 답

Q. 누가 판정했나요?

A. AI가 1차 판정하고 사람이 교차 검토해 확정했습니다. 판정자 간 일치도는 따로 계산하지 않았습니다.

Q. 77%를 전체 SID에 일반화할 수 있나요?

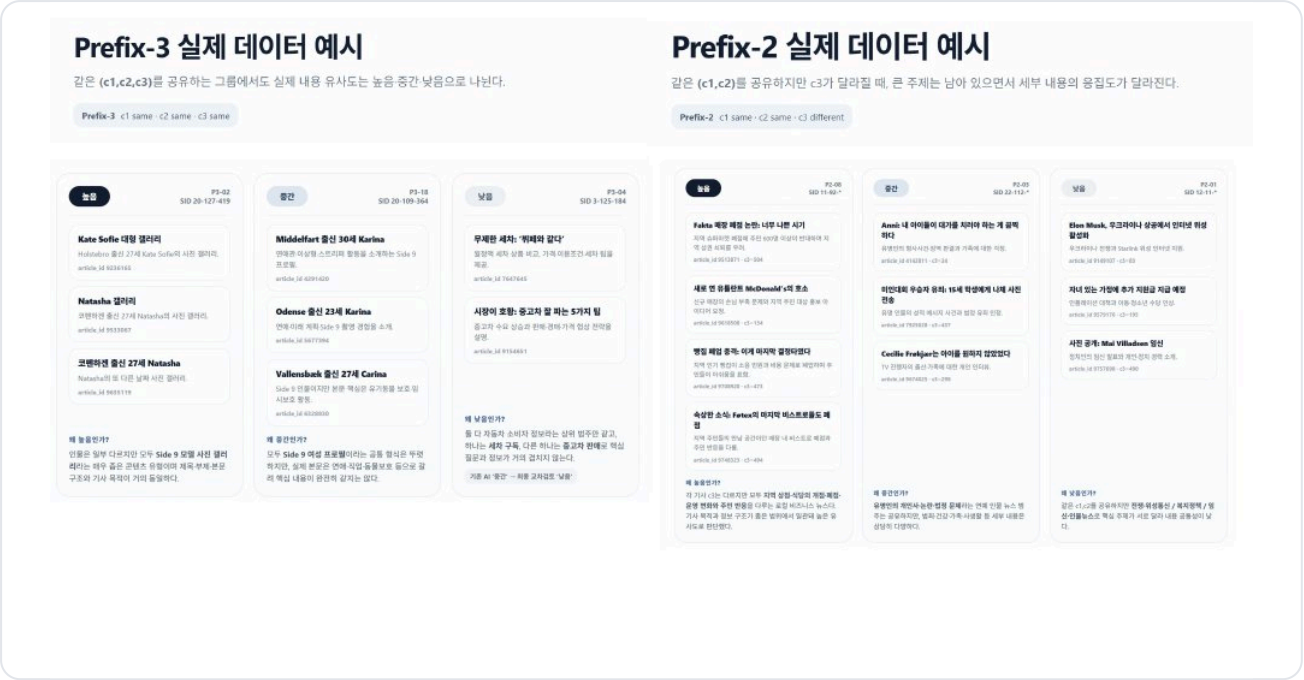
A. 어렵습니다. 큰 그룹을 일부러 많이 뽑은 진단용 표본이라 비율 추정용이 아닙니다. 전체 규모 확인은 9~12장 정량평가가 합니다.

Q. c2 그룹 중에 128개짜리처럼 큰 것도 다 읽었나요?

A. 아닙니다. Prefix-2는 4~10개짜리 그룹만 대상이었습니다.

8장 · Prefix-3 · Prefix-2 실제 데이터 예시

SID 품질 정성



11/05 발표 슬라이드 8

> 같은 c1c2c3 안에서도 높음 · 중간 · 낮음이 갈리고, **c3가 다르면(Prefix-2) 큰 주제만 남고 세부가 흩어집니다.**

1 실험 이유

7장의 판정이 실제로 어떤 모습인지 보여 주는 예시입니다. 숫자만으로는 "비슷하다"가 무슨 뜻인지 와닿지 않아서 넣었습니다.

2 실험 세팅

7장에서 판정한 그룹 중 판정 단계마다 대표 하나씩 골랐습니다(설명용 예시). 실험 세팅은 7장과 같습니다.

3 실험 방식

그룹 안 기사의 제목 · 요약을 나란히 놓고, 왜 그 판정을 받았는지 이유를 붙였습니다.

4 측정 방식

7장과 같은 높음 · 중간 · 낮음 기준입니다.

5 실험 결과

그룹	SID	판정	이유
P3-02	20-127-419	높음	Side 9 사진 갤러리, 형식 · 목적이 거의 같음
P3-18	20-109-364	중간	Side 9 여성 프로필이라는 형식은 같고 본문은 연예 · 직업 · 동물보호로 갈림
P3-04	3-125-184	낮음	"자동차 소비자 정보"라는 큰 범주만 같음 (세차 구독 vs 중고차 판매)
P2-08	11-92-*	높음	지역 상점 · 식당의 개점 · 폐점 뉴스로 일관됨
P2-03	22-112-*	중간	유명인 사생활 · 법정이라는 범주는 같고 세부 주제는 다양
P2-01	12-11-*	낮음	머스크 위성 · 자녀수당 · 연예인 임신으로 서로 다름

c2까지 같으면 큰 주제가, c3까지 같으면 기사 유형과 내용까지 비슷해지는 경향이 보입니다.

발표 한마디 · Prefix-3는 기사 형식과 내용까지 비슷하고, Prefix-2는 큰 주제만 같고 세부가 갈리는 모습이 보입니다.

예상 질문과 답

Q. 왜 형식이 같은 기사가 같은 SID로 묶이나요?

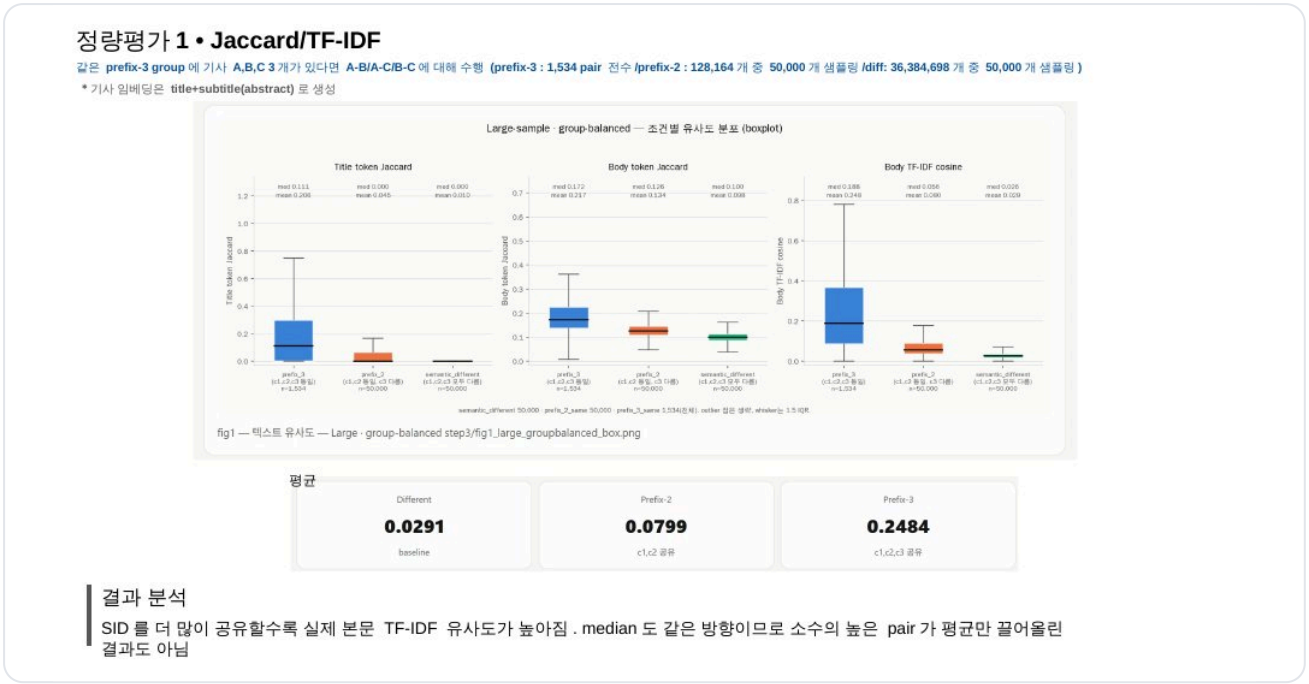
A. SID 입력 임베딩이 제목 + 부제로만 만들어져서, Side 9처럼 제목 틀이 같은 템플릿 기사는 본문이 달라도 비슷해 보입니다.

Q. 예시는 어떻게 골랐나요?

A. 판정 단계마다 대표 하나씩 고른 설명용 예시입니다. 전체 분포는 7장 표와 9장 정량평가가 보여 줍니다.

9장 · 정량평가1 · Jaccard / TF-IDF

SID 품질 정량



11/05 발표 슬라이드 9

> SID를 더 공유할수록 **RQ-VAE가 보지 않은 본문**도 더 비슷합니다. Body TF-IDF cosine 평균 **0.029 → 0.080 → 0.248** (중앙값 0.026 → 0.056 → 0.188).

1 실험 이유

- 정성평가는 표본이 작아서, 같은 질문을 큰 규모의 숫자로 확인했습니다.
- SID는 제목 + 부제 임베딩으로 만들었으므로, **본문**으로 확인하면 RQ-VAE 입력에 없던 정보로 검증하는 셈입니다.

2 실험 세팅

조건	정의	쌍 수
Prefix-3	c1c2c3 같음	가능한 1,534쌍 전부 (그룹에 A, B, C가 있으면 A-B, A-C, B-C)

Prefix-2	c1c2 같고 c3 다름	128,164쌍 중 50,000쌍 샘플링
Different	c1부터 다름	36,384,698쌍 중 50,000쌍 샘플링

대상은 본문이 있는 train 기사 9,338개입니다. 세 지표 모두 **같은 쌍**에서 계산했습니다.

3 실험 방식

1. 조건별로 기사 쌍을 만듭니다(Prefix-2 · Different는 그룹 크기 균형을 맞춰 샘플링).
2. 쌍마다 지표 3개를 계산합니다.
3. 조건별 분포를 상자그림으로 비교하고 평균 · 중앙값을 봅니다.

4 측정 방식

지표	계산
Title Jaccard	두 기사 제목의 단어 집합에서 겹치는 단어 수 ÷ 합친 단어 수
Body Jaccard	같은 계산을 본문 단어로
Body TF-IDF cosine	본문을 단어 점수 벡터로 바꿔 cosine. 점수 = (1 + ln 등장 횟수) × IDF. 흔한 단어는 낮게, 특징 단어는 높게. IDF는 9,338개 본문으로 계산, 60% 이상 기사에 나오는 단어와 1개 기사에만 나오는 단어는 제외

상자그림 읽는 법: 상자 = 가운데 50% 쌍(25~75%), 상자 안 선 = 중앙값, 수염 = 대부분의 범위.

TF-IDF 실제 계산 (코드 `step3/sid_pairs.py`: scikit-learn TfidfVectorizer, sublinear_tf · min_df 2 · max_df 0.6 · L2 정규화, train 본문 9,338개 · 어휘 59,068개). 기사 h3 “Vingegaard i vildt sne-kaos: Løbet stoppet”, 본문 단어 209개:

```
TF  = 1 + ln(그 기사에서 나온 횟수)                          ← 3번 나와도 3배가 아니라 2.1배
IDF = ln((1 + 9,338) / (1 + 그 단어가 나온 기사 수)) + 1   ← 드문 단어일수록 큼
가중치 = TF × IDF → 기사 벡터를 길이 1로 나눔 → 두 기사 벡터의 내적 = TF-IDF cosine
```

단어	횟수	TF	나온 기사 수	IDF	TF × IDF	길이 1로 나눈 값

vingegaard	3	1 + ln 3 = 2.10	28	6.77	14.22	0.241
etape (구간)	3	2.10	84	5.70	11.96	0.203
spanske	2	1 + ln 2 = 1.69	—	4.87	8.25	0.140
efter (~후에)	3	2.10	흔함	1.54	3.23	0.055
at · det · og · i	9 · 8 · 3 · 3	—	60% 넘는 기사	제외	—	—

왜 Jaccard 말고 TF-IDF인가 (축소 예시, 단어 집합만 표시)

```
A (빙에고르): vingegaard, etape, og, i, at, det
B (빙에고르): vingegaard, etape, og, i, sne
C (세금 정책): regering, skat, og, i, at, det
```

	Jaccard	TF-IDF cosine
A–B (같은 주제)	4/7 = 0.57	0.79
A–C (다른 주제)	4/8 = 0.50	0

Jaccard는 모든 단어를 똑같이 세서 og · i · at · det 같은 흔한 단어 겹침만으로 다른 주제도 0.50이 됩니다(stopword를 빼지 않았음). TF-IDF는 흔한 단어를 빼고 특징 단어에 큰 가중치를 줘서 다른 주제는 0, 같은 주제는 높게 나옵니다(0.79는 실제 IDF로 계산). 등장 횟수와 기사 길이도 반영합니다. 실제 결과도 본문 Jaccard 중앙값 0.100 → 0.172(1.7배), TF-IDF 0.026 → 0.188(7배)로 TF-IDF가 차이를 더 뚜렷하게 보여 줍니다. 그래서 Jaccard는 보조 지표, TF-IDF는 핵심 지표입니다.

5 실험 결과

지표	Different	Prefix-2	Prefix-3
Title Jaccard 중앙값 (평균)	0.000 (0.010)	0.000 (0.045)	0.111 (0.206)
Body Jaccard 중앙값 (평균)	0.100 (0.098)	0.126 (0.134)	0.172 (0.217)
Body TF-IDF 중앙값 (평균)	0.026 (0.029)	0.056 (0.080)	0.188 (0.248)

TF-IDF 0.05 미만 쌍	88%	43%	12%

세 지표 모두 Different < Prefix-2 < Prefix-3 순서이고, 평균뿐 아니라 중앙값도 같은 방향이라 일부 높은 쌍이 평균을 끌어올린 결과가 아닙니다.

발표 한마디 · SID는 제목과 부제만 보고 만들었는데, 본문으로 비교해도 SID를 많이 공유할수록 내용이 비슷했습니다.

예상 질문과 답

Q. TF-IDF는 왜 썼나요?

A. Jaccard는 흔한 단어(조사, 접속사 같은 것) 겹침도 세서 차이가 흐려집니다. TF-IDF는 흔한 단어를 깎고 특징 단어를 살려 내용 유사도를 더 잘 보여 줍니다.

Q. 쌍 수가 조건마다 다른데 비교해도 되나요?

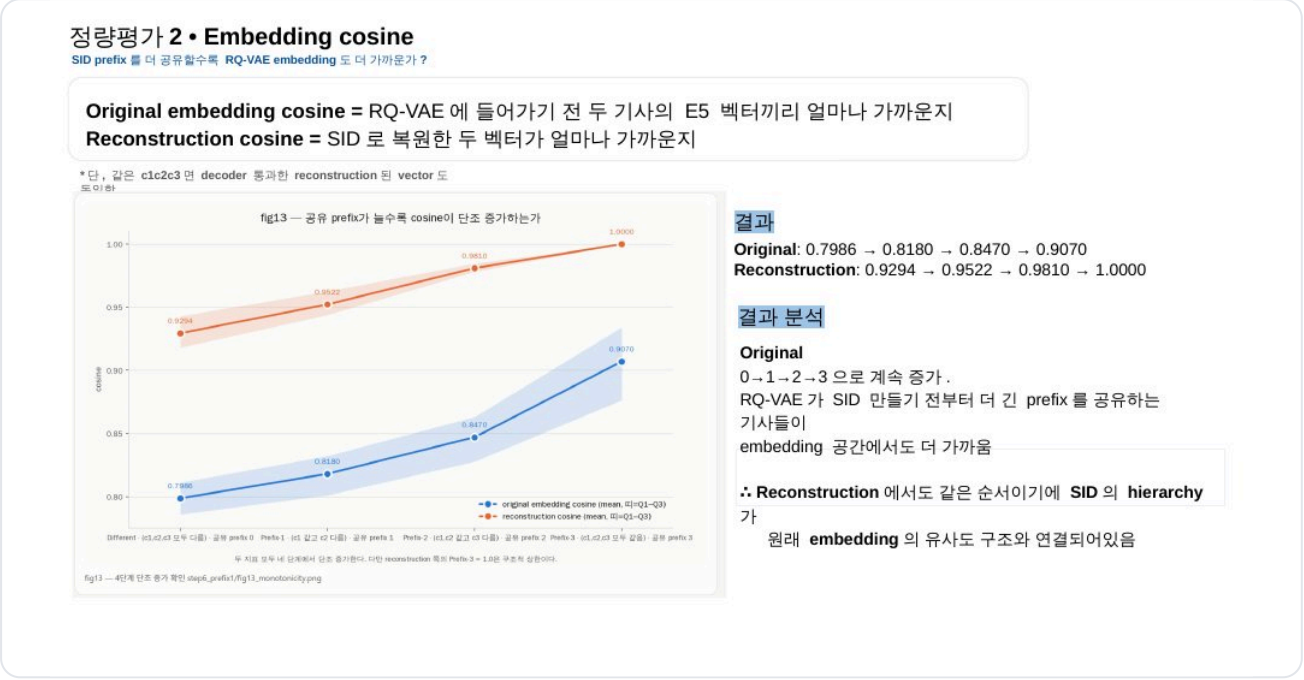
A. 세 조건 모두 1,534쌍으로 맞춘 비교도 했고 순서가 같았습니다.

Q. 0.248이면 높은 건가요?

A. 본문 TF-IDF는 원래 값이 낮은 지표라 기준선(Different 0.029)과 비교합니다. 약 8.6배이고, 상자가 Different 상자와 겹치지 않습니다.

10장 · 정량평가2 · Embedding cosine

SID 품질 정량



> 공유 prefix가 0 → 1 → 2 → 3으로 늘수록 **원본 임베딩도, 복원 임베딩도 계속 가까워집니다**. SID 계층이 원래 임베딩의 거리 구조를 따릅니다.

1 실험 이유

- 9장이 단어 수준이었다면, 여기는 RQ-VAE가 실제로 보는 **임베딩 공간**에서 확인합니다.
- "SID prefix를 더 공유할수록 임베딩도 더 가까운가?"를 원본과 복원 양쪽에서 봅니다.

2 실험 세팅

항목	값
쌍	9장과 같은 쌍 + Prefix-1(c1만 같고 c2 다름) 50,000쌍 추가 → 4단계 (Different · Prefix-1 · Prefix-2 · Prefix-3)
대상	train 기사
비교 벡터	같은 쌍에서 두 종류: ① 원본 E5 ② 각 기사의 SID로 복원한 벡터

3 실험 방식

1. 쌍마다 두 기사의 원본 E5 cosine을 계산합니다(파란 선).
2. 같은 쌍에서 두 기사의 복원 벡터 cosine을 계산합니다(주황 선). 복원 벡터 = decoder(Q1[c1] + Q2[c2] + Q3[c3]).
3. 단계별 평균과 가운데 50% 범위(띠)를 그립니다.

4 측정 방식

cosine = 두 벡터 방향이 얼마나 같은지(1이면 같은 방향). **단, 같은 c1c2c3면 복원 벡터가 완전히 같아서** Prefix-3의 복원 cosine 1.0은 구조상 자동으로 나오는 값입니다(근거에서 제외).

5 실험 결과

	Different	Prefix-1	Prefix-2	Prefix-3

Original cosine (평균)	0.7986	0.8180	0.8470	0.9070
Reconstruction cosine (평균)	0.9294	0.9522	0.9810	1.0000 (구조상)

- 파란 선이 계속 오릅니다. 긴 prefix를 공유하게 된 기사들은 RQ-VAE가 SID를 만들기 **전부터** 임베딩 공간에서 가까웠습니다.
- 주황 선도 같은 순서라, SID만 남겨도 그 거리 구조가 유지됩니다.
- 주황 선이 전부 높은 것은 코드 3개로 압축했다가 펴면서 벡터들이 서로 가까워진 효과입니다.

발표 한마디 · SID를 더 공유한 기사일수록 원래 임베딩에서도 가까웠고, 복원한 벡터에서도 같은 순서가 유지됐습니다.

예상 질문과 답

Q. Different인데도 0.80이면 높은 것 아닌가요?

A. E5는 모든 문장 벡터가 좁은 방향에 몰려 있어서 무관한 기사끼리도 0.77~0.83이 나옵니다. 절대값이 아니라 단계별 증가를 봅니다.

Q. 파란 선과 주황 선은 같은 쌍인가요?

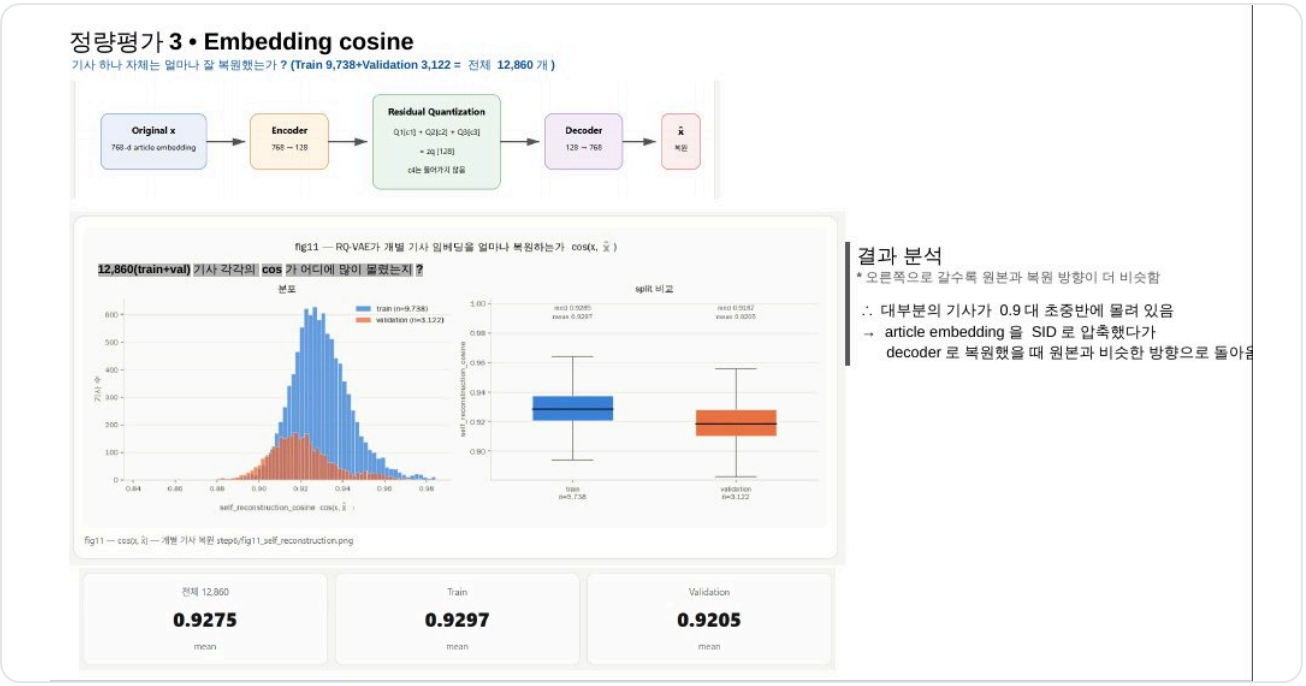
A. 네. 같은 쌍에서 비교하는 벡터만 다릅니다.

Q. 0.93이면 93% 복원인가요?

A. 아닙니다. cosine은 퍼센트가 아니고, 이 값은 서로 다른 두 기사의 복원 벡터끼리 비교한 값입니다.

11장 · 정량평가3 · 기사 하나의 자기 복원

SID 품질 정량



11/05 발표 슬라이드 11

> 기사 하나를 SID 세 자리로 압축했다가 되살려도 원본과 같은 방향으로 돌아옵니다. 평균 **0.9275**, 가장 낮은 기사도 0.842.

---

1 실험 이유

- 9 · 10장이 기사 쌍이었다면 여기는 **기사 하나**가 SID로 얼마나 잘 표현되는지 봅니다.
- train(학습 때 본 기사)과 validation(처음 보는 기사)을 나눠, 새 기사에도 SID가 잘 붙는지 확인합니다.

---

2 실험 세팅

항목	값
대상	추천 기록에 쓰인 기사 전부 12,860개 = train 9,738 + validation 3,122
모델	UNI RQ-VAE (epoch 180)

---

3 실험 방식

1. 원본 E5(768) → encoder → 128차원
2. 양자화로 SID (c1, c2, c3)를 받고, 세 코드북 벡터를 더함 (c4는 decoder에 들어가지 않음)
3. decoder로 다시 768차원으로 복원
4. 원본 E5와 복원 벡터의 cosine을 기사마다 계산

4 측정 방식

기사마다 숫자 1개(자기 복원 cosine)가 나옵니다. 히스토그램은 분포 모양(어디에 몰렸나), 상자그림은 요약(중앙값 · 가운데 50%)을 보여 주며 둘 다 같은 숫자입니다. 오른쪽으로 갈수록 원본과 복원 방향이 더 비슷합니다.

5 실험 결과

	전체 12,860	Train 9,738	Validation 3,122
평균	0.9275	0.9297	0.9205
중앙값	0.926	0.9285	0.9187
하위 25% / 상위 25%	0.918 / 0.936	—	—
최저 / 최고	0.842 / 0.984		

- 대부분 기사가 0.92~0.94에 몰려 있고 크게 실패한 기사는 없습니다.
- validation이 train보다 조금 낮지만(0.9205 vs 0.9297) 차이가 작아 새 기사에도 SID가 잘 붙습니다.
- 비교: 각 기사와 **가장 비슷한 다른 기사**의 E5 cosine 평균은 0.905, 아무 기사 2개는 0.800입니다. 복원 벡터(0.928)는 가장 비슷한 다른 기사보다도 자기 원본에 더 가깝습니다.

발표 한마디 · 기사 하나를 SID로 압축했다 되살려도 평균 0.93으로 원래 방향으로 돌아오고, 새 기사도 거의 같은 수준입니다.

예상 질문과 답

Q. 이 점수가 좋은 건가요?

A. E5는 무관한 기사끼리도 0.80이 나와 절대값보다 비교가 중요합니다. 복원 벡터는 가장 비슷한 다른 기사(0.905)보다 자기 원본에 더 가까워(0.928) “그 기사다운” 방향이 대체로 남아 있습니다.

Q. 왜 validation이 조금 낮나요?

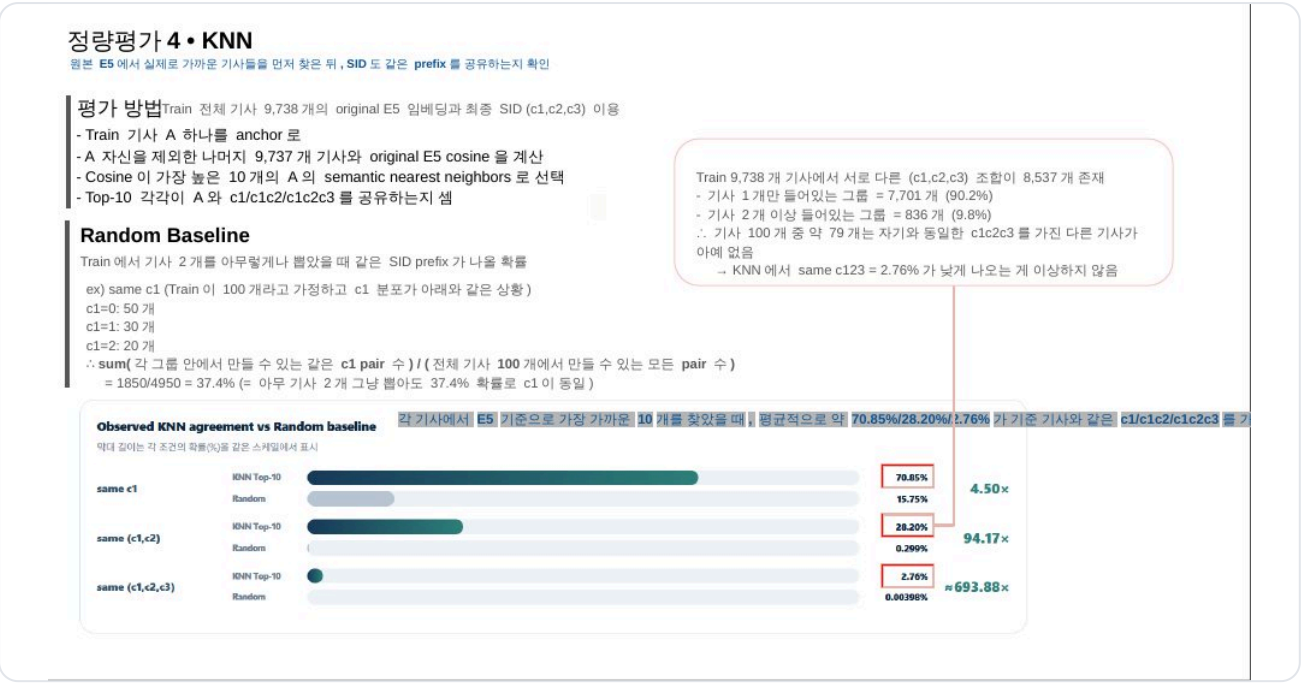
A. 학습 때 보지 않은 기사이고, validation 기사는 사건 코드(사건 기사 평균으로 정한 c2)를 물려받기 때문입니다.

Q. validation 숫자는 믿을 만한가요?

A. checkpoint를 validation 복원 loss로 골랐기 때문에 약간 낙관적일 수 있습니다. 고른 것이 epoch 하나라 영향은 작습니다.

12장 · 정량평가4 · KNN

SID 품질 정량



11/05 발표 슬라이드 12

> 방향을 뒤집어도 성립합니다. 원본 E5에서 가장 가까운 이웃 10개 중 **약 7개가 같은 c1, 3개가 같은 c1c2**이고, 무작위의 4.5배 · 94배입니다.

1 실험 이유

- 9~11장은 "SID가 같으면 가까운가"였고, 여기는 반대로 "**원래 가까운 기사끼리 SID도 같은가**"입니다.
- 두 방향이 모두 성립해야 SID가 임베딩의 이웃 구조를 보존한다고 말할 수 있습니다.

2 실험 세팅

항목	값
대상	train 기사 9,738개의 원본 E5와 최종 SID (c1, c2, c3)

이웃 수	기사마다 가장 가까운 10개 (K = 10) → 9,738 × 10 = 97,380쌍

3 실험 방식

1. train 기사 A 하나를 기준으로 잡습니다.
2. A를 뺀 나머지 9,737개와 원본 E5 cosine을 계산합니다.
3. cosine이 가장 높은 10개를 A의 이웃으로 고릅니다.
4. 이웃 10개 각각이 A와 같은 c1 · c1c2 · c1c2c3를 갖는지 셉니다.
5. 9,738개 기사 모두 반복해 비율을 냅니다.

4 측정 방식

숫자만으로는 높은지 알 수 없어서 **무작위 기준**과 비교합니다. 무작위 기준 = train에서 아무 기사 2개를 뽑았을 때 같은 prefix일 확률 = (그룹마다 그 안에서 만들 수 있는 쌍 수의 합) ÷ (전체 쌍 수).

```
예: 기사 100개, c1=0이 50개 · c1=1이 30개 · c1=2가 20개
같은 c1 쌍 = 50×49/2 + 30×29/2 + 20×19/2 = 1,225 + 435 + 190 = 1,850
전체 쌍   = 100×99/2 = 4,950        → 무작위 기준 = 1,850 / 4,950 = 37.4%
```

5 실험 결과

기준	E5 이웃 10개 중 일치	무작위 기준	배수	이론상 최대 (달성률)
같은 c1	70.85%	15.75%	4.5×	99.8% (71%)
같은 c1c2	28.20%	0.299%	94×	85.2% (33%)
같은 c1c2c3	2.76%	0.004%	694×	3.86% (72%)

- 세 기준 모두 무작위보다 훨씬 높습니다.
- c1c2c3 2.76%가 낮아 보이지만, train 기사에서 서로 다른 (c1,c2,c3) 조합이 8,537개이고 그중 7,701개(90.2%)가 기사 1개짜리입니다. 즉 기사의 약 79%는 같은 c1c2c3를 가진 다른 기사가 아예 없어서, 이웃을 완벽히 골라도 최대 3.86%입니다. 최대치의 72%를 달성했습니다.

발표 한마디 · 원래 임베딩에서 가까운 이웃 10개 중 약 7개가 같은 c1, 3개가 같은 c1c2를 가져서 무작위보다 4.5배, 94배 높았습니다.

예상 질문과 답

Q. 새 기사(validation)에서도 그런가요?

A. 네. validation 3,122개 안에서 같은 방법으로 보면 c1 65.41%, c1c2 21.04%로 train 같은 크기 표본(65.41%, 22.37%)과 비슷합니다.

Q. E5는 RQ-VAE 입력인데 자기 확인 아닌가요?

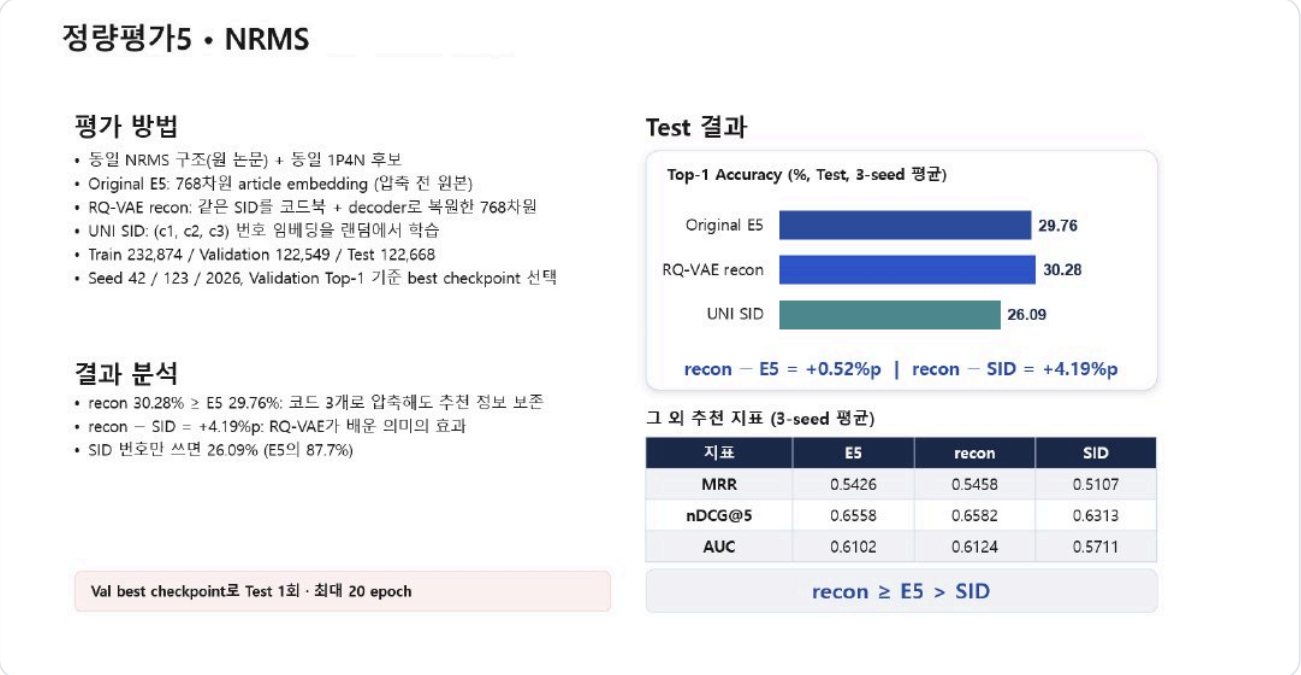
A. 맞습니다. 이 실험은 입력 공간의 이웃 구조를 SID가 얼마나 보존하나를 봅니다. 입력에 없던 본문으로 확인한 9장과 함께 봐야 합니다.

Q. c1이 30% 어긋나는 이유는?

A. c1은 편집 카테고리로 고정한 값이라, E5로는 비슷해도 편집부가 다른 카테고리로 분류한 기사입니다.

13장 · 정량평가5 · NRMS

추천 성능 Test



11/05 발표 슬라이드 13

> 표준 추천 모델 NRMS에서 기사 입력만 바꾸면 **RQ-VAE recon 30.28% ≥ 원본 E5 29.76% > SID 번호만 26.09%**. SID가 추천 정보를 잃지 않았고, RQ-VAE가 배운 의미까지 쓰면 E5 수준입니다.

1 실험 이유

- 7~12장은 SID가 기사 내용을 잘 담는지였고, 여기는 **추천에 필요한 정보가 남는지**를 추천 성능으로 확인합니다.
- 검증된 추천 모델(NRMS)을 완전히 고정하고 기사 입력만 바꾸면, 성능 차이는 전부 "기사를 어떻게 표현했나"의 차이가 됩니다.
- recon 모드는 교수님 제안입니다: NRMS 뉴스 입력을 랜덤 초기화 대신 RQ-VAE 출력(의미가 담긴 임베딩)으로.

2 실험 세팅

항목	값
모델	NRMS 원 논문 구조 (정민교 님 코드, EB-NeRD 읽는 부분만 수정)
구조	기사 인코더(모드별) → 사용자 인코더: self-attention 20 head + additive attention 300 → 200 → 1 → 사용자 벡터 u (300차원) → 점수 = u · 후보 벡터
학습	후보 5개 점수에 softmax → −log P(정답) (원 논문과 같은 방식) · Adam lr 1e-3 · batch 64 · dropout 0.2 · **최대 20 epoch**
데이터	1P4N (V1과 같은 파일): train 232,874 / validation half 122,549 (best epoch 선택) / test 122,668 (1회)
seed	42 · 123 · 2026, 3개 평균

모드	기사 → 300차원	학습되는 것
Original E5	원본 E5 768 → Linear 768 → 300	Linear만 (E5 고정)
RQ-VAE recon	SID → RQ-VAE 코드북 Q1[c1]+Q2[c2]+Q3[c3] → decoder → 768 → Linear → 300	Linear만 (recon 고정)
UNI SID	E1[c1] + E2[c2] + E3[c3] (번호마다 300차원 번호표)	번호표 자체를 랜덤에서 학습

recon과 SID는 **같은 SID 정보**입니다(같은 SID면 같은 벡터, 서로 다른 벡터 10,952개). 차이는 번호의 뜻을 RQ-VAE에서 가져오느냐(recon), 처음부터 배우느냐(SID)뿐입니다.

3 실험 방식

NRMS 코드는 바꾸지 않고, 모드마다 기사 입력 표만 바꿔 학습합니다. recon 표는 UNI RQ-VAE로 기사 12,860개의 복원 벡터를 미리 계산한 것으로, 정량평가 3의 복원 벡터와 완전히 같음(차이 0)을 확인했습니다. epoch이 끝날 때마다 validation Top-1을 재 가장 좋은 epoch을 저장하고, 그 checkpoint로 test를 한 번 채점합니다.

4 측정 방식

Top-1 · MRR · nDCG@5 · AUC (6장과 같은 정의). seed 3개 평균.

5 실험 결과

Test 지표	Original E5	RQ-VAE recon	UNI SID
Top-1	29.76%	30.28%	26.09%
seed별 (42 / 123 / 2026)	29.65 / 28.81 / 30.82	30.51 / 30.30 / 30.04	26.20 / 26.01 / 26.05
MRR	0.5426	0.5458	0.5107
nDCG@5	0.6558	0.6582	0.6313
AUC	0.6102	0.6124	0.5711
best epoch	9 / 5 / 4	10 / 7 / 19	2 / 1 / 3

- **recon − E5 = +0.52%p** (E5 seed 편차 1.0보다 작음): 768차원을 코드 3개로 압축해도 추천 정보 손실이 없습니다.
- **recon − SID = +4.19%p**: 같은 SID라도 RQ-VAE가 배운 의미(코드북 · decoder)를 쓰면 오릅니다. SID 번호만 쓰면 그 의미를 버리고 처음부터 배워야 합니다.
- SID는 세 seed 모두 epoch 1~3에서 최고라 epoch을 늘려도 그대로입니다 → 차이는 학습 부족 때문이 아닙니다.
- 5 epoch으로 돌렸을 때도 결론이 같았습니다(E5 29.92 · recon 29.99 · SID 26.09).

발표 한마디 · 교수님 말씀대로 NRMS 입력을 RQ-VAE 복원 벡터로 바꾸니 30.28%로 원본 E5(29.76%)와 같은 수준이 나왔고, SID 번호만 쓴 경우보다 4.19%p 높았습니다. 우리 SID가 추천에 필요한 의미를 보존한다고 볼 수 있습니다.

예상 질문과 답

Q. SID 번호에 정말 의미가 있나요?

A. SID를 기사끼리 무작위로 섞은 대조군은 21.13%로 거의 무작위 수준입니다. SID는 26.09%라 번호 배정 자체에 의미가 있습니다. (슬라이드에는 없고 답변용)

Q. SID도 RQ-VAE가 만든 건데 recon과 뭐가 다른가요?

A. RQ-VAE가 만든 것은 ① 번호 배정과 ② 번호의 뜻(코드북 벡터 + decoder) 두 가지입니다. SID 모드는 ①만 쓰고, recon 모드는 ①과 ② 모두 씁니다.

Q. NRMS 학습 방식(후보 5개 중 고르기)은 우리 V1과 같은 것 아닌가요?

A. 네, 학습 목표는 같습니다(NRMS 원 논문 방식). 그래서 공정하게 비교할 수 있고, V1의 차이는 기사를 SID 토큰열로 표현하고 후보를 계층별 확률로 채점한다는 점입니다.

Q. recon이 V1(29.46%)보다 높은데요?

A. recon은 RQ-VAE 벡터까지 쓰는 경우라 입력이 다릅니다. SID 번호만 쓰는 조건에서는 NRMS 26.09%, V1 27.07%, V1 + level 29.46%로 V1이 높습니다.

Q. recon을 학습 중에 업데이트하는 버전은요?

A. 이번에는 recon을 고정 입력으로 넣었습니다. 초기값으로 두고 업데이트하는 버전이 필요할지 여쭤보고 싶습니다.

부록 · RQ-VAE vs Transformer V1 비교 (학습 · 선택 · 결과)

정리

> 둘 다 **① 설정 선택 + ② epoch 선택** 두 층으로 골랐고, 둘 다 **validation만** 썼습니다. 차이는 ①을 RQ-VAE는 GPT가, Transformer는 코드 규칙이 판단했다는 점입니다.

1. 역할과 데이터

항목	RQ-VAE (UNI)	Transformer V1
하는 일	기사 E5 → SID (c1, c2, c3, c4) 만들기	사용자 기록 SID → 후보 5개 중 다음 클릭 고르기
입력	기사 하나의 E5 768차원 (제목 + 부제)	기록 최근 50개 기사의 c1 c2 c3 c4 토큰 + 후보 5개의 SID
출력	복원 벡터 $\hat{x}$ (768) + SID 번호	후보 5개 점수 S = L1 + L2 + L3

학습 데이터	train 기사 9,738개	1P4N train 232,874문제
validation	validation 기사 3,122개	validation half 122,549문제 (validation 주 앞쪽 절반)
test	없음 (클릭 예측 모델이 아님). SID 품질은 7~12장에서 따로 평가	122,668문제 (validation 주 뒤쪽 절반), 마지막에 1회

2. 모델과 학습

항목	RQ-VAE (UNI)	Transformer V1
구조	encoder 768→512→128 → 양자화 → decoder 128→512→768	T5식 encoder-decoder
크기	잠재 128 · c1 25(카테고리 고정) · Q2 128 · Q3 512	d_model 256 · head 8 · 2층 · d_ff 1024
핵심 방식	잔차 양자화 (STE, 정규화 없음, K-means 초기화)	level별 임베딩 · 출력 head, SEP 미사용, dropout 0
loss	복원 1 + codebook 0.25 + commitment 0.1 + uniqueness 0.05 (margin 0.5)	후보 5개 점수에 listwise cross-entropy
optimizer	lr 2e-4 · batch 256 · weight decay 0 (UNI 계열 gin 기준)	AdamW lr 5e-5 · batch 128 · weight decay 0
1 epoch 걸음 수	약 39 step	약 1,820 step
최대 epoch	1,283	30
평가 간격	10 epoch마다	매 epoch
early stopping	200 epoch 이후, 평가 20번 연속(약 200 epoch) 개선 없으면 정지 (개선 = 0.001 넘게 좋아짐)	5 epoch 연속 개선 없으면 정지
seed	42	42 (최종 상위 3개는 42 · 123 · 2026으로도 확인)

epoch 수가 크게 다른 이유: epoch은 데이터를 한 바퀴 도는 단위라 데이터 크기에 따라 길이가 다릅니다. RQ-VAE 180 epoch(≈ 7,020 step)과 Transformer 4 epoch(≈ 7,280 step)이 걸음 수로는 비슷합니다. RQ-VAE는 복원 loss가 천천히 오래 줄어 최대값을 넉넉히 잡았고,

Transformer는 epoch 1에서 이미 validation 최고라 30이면 충분했습니다(84 run을 돌려야 하는 시간 제약도 있음).

---

3. 선택 방법

항목	RQ-VAE (UNI)	Transformer V1
① 설정 선택 지표	복원 loss ↓ · c123 충돌 ↓ · c2 사건 일관성 ↑ · Q2/Q3 사용률 ↑ · ΔC2/ΔC3 ↑	Top-1 → MRR → nDCG@5 → AUC → Preference Loss → Positive Probability 순서
① 판단 방식	단계별 결과표를 GPT가 종합 판단	코드에 고정된 규칙: 우선순위 + 허용 범위(Top-1 0.5%p 등), 동률이면 작은 모델
① 탐색 구조	코드북 → 잠재 · hidden → 양자화 → loss · 학습 단계, 마지막 weight decay 단계 001번 = WD-001 → uniqueness loss 추가 = UNI	6단계 × 단계마다 상위 2개 = 84 run, 마지막 상위 3개 seed 재확인
① 재현성	같은 판단이 나온다는 보장 없음	같은 결과표면 항상 같은 선택
② epoch 선택 지표	validation 복원 loss — 가장 낮은 epoch	validation Top-1 — 가장 높은 epoch
② 적용 범위	모든 run마다 자동 (`checkpoint_best_rec.pt`)	모든 run마다 자동 (`checkpoint_best`)
② 결과	epoch 180 (≈ 7,020 step)	epoch 1 (≈ 1,820 step)
best epoch 해석	천천히 오래 개선	첫 epoch 뒤 바로 과적합 (test 정답 대부분이 새 기사)
test를 선택에 썼나	안 씀	안 씀
주의	같은 run에 total loss 기준 best(epoch 20)도 저장되지만 쓰지 않음	탐색은 후보 순서를 섞기 전 데이터, 최종은 섞은 데이터로 재학습

```
RQ-VAE
설정 A 학습 → 10 epoch마다 validation 복원 loss → 최저 epoch 저장(best_rec) ┐
설정 B 학습 → 같은 방식 → best_rec                                          ├→ 각
best_rec로 SID → 5개 지표 표 → GPT 판단 → 다음 단계
   … 단계 반복 … → 최종(UNI) best_rec = epoch 180 → 최종 SID
```

```
Transformer V1
설정 8개 학습 → 매 epoch validation Top-1 → 최고 epoch 저장 → 규칙으로 비교 → 상위 2개
   … 6단계 84 run … → 상위 3개 seed 확인 → 최종 설정을 섞은 데이터로 재학습 → best
epoch 1 → test 1회
```

---

4. 최종 결과

항목	RQ-VAE (UNI)	Transformer V1
핵심 결과	c123 충돌 24.39% (WD-001 31.56%)	test Top-1 27.07% (무작위 20%)
그 밖의 지표	validation 복원 loss 0.1525 · c2 사건 일관성 74.15% · Q3 512/512 · max c4 16	MRR 0.518 · nDCG@5 0.637 · AUC 0.579 · validation Top-1 27.06%
지표 기준 데이터	복원 loss는 validation 3,122개 / 충돌 · 일관성 · 사용률은 전체 12,860개	모두 test 122,668문제
후처리 개선	—	level 가중치 (1, 0.5, 0.1), validation에서 선택 → test 29.46%
코드 위치	`RQVAE/train_rqvae.py` (Colab에서 학습)	`minji1217/sid_project-transformer` 의 `Transformer/` (탐색: `sweep/`)

확인 필요 · RQ-VAE 설정 탐색의 정확한 단계 수 · 단계별 후보값 · GPT 판단 근거는 기록이 없습니다. 발표에서 물으면: "단계마다 다섯 지표를 표로 정리해 종합 판단했고 판단 보조로 LLM을 활용했습니다. 각 설정의 checkpoint는 validation 복원 loss 최저 epoch으로 자동 선택했습니다. 판단 규칙을 명시적으로 정해 재검증하는 것은 다음 과제입니다."

공통 예상 질문

Q. test는 어떻게 만들었나요?

A. EB-NeRD는 정답이 공개된 test가 없어서 validation 주의 클릭 기록을 시간순으로 반으로 나눴습니다. 앞쪽 절반(122,549문제)은 설정 · checkpoint 선택에만, 뒤쪽 절반(122,668문제)은 성적 보고에만 씁니다.

Q. 1P4N이 뭔가요?

A. 노출 한 번에서 클릭한 기사 1개(정답)와 보고도 클릭하지 않은 기사 4개(오답)로 문제 하나를 만든 것입니다. 정답과 c1c2c3가 같은 오답은 빼고, 오답이 4개가 안 되면 그 클릭은

버립니다. 무작위 Top-1은 20%입니다.

Q. 사용자는 어떻게 표현하나요?

A. 사용자 ID 임베딩은 없고, 그 impression 시점까지 읽은 기사(running history, 최근 50개)로 매번 표현합니다. 같은 사용자도 impression마다 history가 달라집니다.

Q. seed가 장마다 다른가요?

A. NRMS(13장)는 seed 3개 평균, Transformer V1(5 · 6장)은 seed 42 한 번입니다. seed 하나짜리는 1%p 안쪽 차이를 같은 수준으로 봅니다.

Q. 원래 설계와 지금 SID는 같은가요?

A. 두 군데가 다릅니다. ① 학습 중 c2를 사건 단위가 아니라 기사마다 고릅니다(사건 일관성 74%). ② 새 사건의 c2를 사건 기사 전체 평균으로 정해 일부(1.07%)에서 미래 기사 정보가 들어갑니다. 여민 님이 고치는 중입니다.

Q. 다음 단계는?

A. ① V1의 SID 토큰 임베딩을 RQ-VAE 코드북으로 초기화(13장 recon 결과에서 착안) ② c2를 원래 설계대로 사건 단위로 ③ c3의 새 기사 일반화 개선(6장 level 가중치 결과) ④ TIGER와 공정하게 비교하는 방법 정하기.

근거: `experiments/nrms_original_ebnerd_v2/`, `experiments/nrms_recon_ebnerd_v1/`, `experiments/nrms_epochs20_ebnerd_v1/`, `experiments/knn_e5_sid_v1/`, `experiments/uni_v1_test_eval/`, `reports/NRMS_그림해설.html`, `reports/V1_트랜스포머_그림해설.html`